Sprint 3 Documentation – Project Setup, REST API, DAL

Team: Sebastian and Felix

Repository: https://github.com/BunnySupreme/SWEN3Project

Sprint duration: 04.11.2025 – 12.11.2025

Estimated workload: 10 h / person / week

1. Sprint Goal (from assignment)

- Set up RabbitMQ
- Integrate Queues into REST server
- Set up "empty" OCR worker for processing uploads (will consume messages from input queue, but will produce only static results for the result queue)
- Failure/exception handling (with layer-specific exceptions) implemented
- Use logging in remarkable/critical positions

2. Implemented Functionality

2.1 RabbitMQ

- A new RabbitMQ service was added to the docker-compose
- The REST server produces messages for an input queue which the OCR worker can consume
- The REST server consumes messages received via a results queue from the OCR worker

2.2 OCR Worker

- A new OCR worker service was added to the docker-compose
- The OCR worker consumes messages from the input queue. In later sprints, it will use the message's information to fetch the file associated with the id provided via the message from the MinIO file storarge
- The OCR worker sends a static summary into the results queue. In later sprints, it will send produce this summary by extracting the text content from the file and sending it to a GenAI worker for analysis.

2.3 Failure/Exception Handling

- In Sprint 2, exception handling middleware for unhandled exceptions had already been implemented in the REST server. Unhandled exceptions would also be logged using a log4net logger
- Also in Sprint 2, some top-level status information was logged using another log4net logger
- Log4net loggers were now also added to all services (DocumentService, RabbitProducerService, and RabbitConsumerService) to track their behavior
- RabbitConsumerService features connection retry logic, with each failed connection attempt being logged

3. Architecture (Sprint 3)

3.1 Frontend

- Presenation layer: Vue.js/Quasar app

3.2 REST server

- API layer: DocumentsController
- Business layer: DocumentService, RabbitProducerService, RabbitConsumerService
- Data access layer: DocumentRepository

3.3 OCR Worker

- API layer:
- Business layer:

3.4 Database

- Database: PostgreSQL container from docker-compose.yml

Refer to Architecture.md for diagrams.

4. Testing

Unit tests implemented in project Paperless.UnitTests:

- DocumentsControllerTests

- DocumentServiceTests
- DocumentServiceValidationTests

Database, repositores, validators, mappers, producers and services mocked using Moq, each where required

All tests passing at the end of Sprint 3

5. Problems

RabbitConsumerService would cause exceptions during startup of the REST server which after a few retries of starting the server would resolve on their own. This was due to RabbitMQ not yet being ready to handle the service's connection attempts despite the Docker service container having started successfully. After this successful start, RabbitMQ still needed a few seconds to be ready for accepting connections. Thus, connection retry logic was added to the service, with a maximum of ten attempts being allowed. This way, the service would only throw an exception if all ten attempts of connecting to RabbitMQ had failed, which resolved the issue.

6. Workload (Estimate)

Team Member	Week / Sprint	Estimated Hours	Main Tasks
Sebastian	Sprint 1	10	CI/CD pipeline, docker compose yaml, Queues in REST server, Logging/Exception handling
Felix	Sprint 1	10	CI/CD pipeline, docker compose yaml, Vue.js/Quasar App, OCR Worker